



## سوال (۱)

(الف)

استفاده از متد Chunked-IO هزینه IO را کاهش می دهد. زیرا در این روش صفحات پشت هم بصورت block خوانده میشود، هزینه IO نسبت به حالتی که به تعداد صفحات که در جاهای گوناگون دیسک وجود داشت و IO انجام می دادیم کمتر می شود. اما از طرف دیگر با افزایش سایز block ها ، تعداد passها زیاد می شود و علت آن این است که fan-out کم میشود.

افزایش تعداد passها، افزایش هزینه ورودی/خروجی را در پی دارد. وقتی سایز block ها خیلی زیاد شود ممکن است افزایش passها اثر بیشتری بر روی IO بگذارد و به صرفه نباشد. پس بطور کلی یک trade-off بین افزایش سایز بلوک ها( که سبب کاهش IO می شود) و افزایش passها ( که سبب کاهش IO می شود) وجود دارد.

(ب) قسمت DISTINCT هزینه بر است.

بخش های مختلف عملگر projection به شرح زیر است:

- Hashing: در این روش داده ها را partition می کنیم، سپس در مموری با دستور in-memory-hash دیتای duplicate را حذف می کنیم.
- Sorting: ابتدا سورت می کنیم و بر همین اساس duplication ها را حذف می کنیم.

(ج) خیر. برای انجام عملیات sort merge join، لازم است که نتایج میانی عملیات سورت در محلی ذخیره شوند تا امکان ترکیب و سورت مجدد آن ها فراهم شود. اما در طرح fully pipelined، این ذخیره سازی انجام نمی شود و فرآیند به صورت پیوسته و بدون توقف ادامه می یابد.

(د)

(سوال ۲)   
 استفاده از هند

chained-IO هزینه IO کمتر

$$2N \times (1 + \log_{B-1}^{[N/B]})$$

: external merge sort هزینه

$$2N \times (1 + \log_{B-1}^{[N/2B]})$$

: Internal merge sort هزینه

$$\hookrightarrow 2N \times (1 + \log_{B-1}^{[N/B]}) = 2N \times (1 + \log_{B-1}^{[N/2B]})$$

$$\log_{B-1}^{[N/B]} = \log_{B-1}^{[N/2B]} = 0$$

$$[N/B] = [N/2B] = 1$$

$$N/B \leq 1 \quad \& \quad N/2B = 1$$

$$\hookrightarrow \log_{B-1}^{[N/B]} = \log_{B-1}^{[N/2B]} = \frac{N}{2B} = \frac{N'}{B} \rightarrow \boxed{\frac{N'}{B} = \frac{1}{2} \frac{N}{B}}$$

\$N \leq B\$ است، بنابراین در هر block 1 رکورد قرار می‌گیرد. در هر pass فقط یک رکورد منتقل می‌شود!

## سوال ۲

(a) از آنجا که term های کوثری ی پریفیکسی از کلیدهای شاخص هستند، می‌توان از شاخص درختی استفاده نمود. برای استفاده از شاخص هش، باید تمامی کلیدهای شاخص مقادیر دقیقی داشته باشند، بنابراین در اینجا امکان استفاده از شاخص هش وجود ندارد.

(b)  $a = 1 \text{ AND } (b < 7 \text{ OR } c = 2)$

در روش hash به دلیل آن که از search range پشتیبانی نمی‌شود، و به دلیل آن که "یا-OR" داریم، نمی‌توان از این شاخص استفاده کرد.

در روش B+ Tree قسمت  $a=1 \text{ AND } b<7$  از ایندکس قابل جداسازی و استخراج است. ولی بخش  $a=1 \text{ AND } c=2$  که باید عملیات OR با قسمت قبل روی آن انجام شود، قابل دسترسی نیست. زیرا پریفیکسی از کلید شاخص نیست.

عبارت سمت چپ به دلیل اینکه ترم‌های آن پیشوندی از کلیدهای شاخص هستند، قابل استفاده با شاخص درخت است. اما برای عبارت سمت راست نمی‌توان از این شاخص استفاده کرد. بنابراین به طور کلی نمی‌توان از شاخص درخت بهره برد. البته می‌توان با استفاده از شاخص درخت، رکوردهای  $a \neq 1$  را از فضای جستجو حذف کرده و در میان رکوردهای باقی‌مانده به اسکن پرداخت.

(c) ابتدا باید کوئری داده شده را تجزیه کنیم:

$(a = 1 \parallel b < 7) \&\& (a = 1 \parallel c = 2)$

با توجه به دلایلی که قبلاً ذکر شد، نمی‌توان از هیچ شاخصی استفاده کرد. تنها برای عبارت سمت چپ می‌توانستیم از شاخص درخت استفاده کنیم، اما چون برای عبارت سمت راست نمی‌توان این کار را انجام داد، استفاده از شاخص درخت بی‌فایده است.

### سوال (۳)

(الف)

#### سناریو اول:

در این سناریو، شاخص‌های مورد بررسی شامل شاخص درختی clustered، شاخص درختی unclustered و شاخص هش clustered هستند.

محاسبات هزینه‌ها به شرح زیر است:

- هزینه شاخص هش clustered:  $0.1 * 0.1 * 100 = 1$

- هزینه شاخص درختی unclustered:  $0.1 * 20000 = 2000$

- هزینه شاخص درختی clustered:  $0.1 * 100 = 10$

- هزینه اسکن کامل: ۱۰۰

به دلیل وجود فاکتور کاهش بیشتر در حالتی که هر دو شاخص clustered باشند، استفاده از شاخص هش را به شاخص درختی ترجیح می‌دهیم.

#### سناریو دوم:

در این سناریو، شاخص هش (hash index) به صورت unclustered مورد بررسی قرار دارد، در حالی که شاخص درخت (tree index) به صورت clustered است.

محاسبات هزینه‌ها به شرح زیر است:

- هزینه شاخص هش  $unclustered: 0.1 * 0.1 * 20000 = 200$

- هزینه شاخص درخت  $clustered: 0.1 * 100 = 10$

- هزینه اسکن کامل: ۱۰۰

در این حالت، استفاده از شاخص درخت clustered بهتر است، زیرا بر اساس محاسبات فوق، برای انجام ۱۰ عملیات I/O نیاز داریم، در حالی که استفاده از شاخص هش unclustered به دلیل نیاز به ۲۰۰ عملیات I/O برای هر مقدار جستجو، کارایی پایین‌تری دارد.

بنابراین، درخت clustered بهترین انتخاب است زیرا به کمترین هزینه I/O نیاز دارد و عملکرد بهتری دارد در مقایسه با شاخص هش unclustered.

#### سناریو سوم:

هر دو شاخص به صورت unclustered در نظر گرفته شده‌اند.

محاسبات هزینه‌ها به شرح زیر است:

- هزینه شاخص هش  $unclustered: 0.1 * 0.1 * 20000 = 200$

- هزینه شاخص درخت  $unclustered: 0.1 * 20000 = 2000$

- هزینه اسکن کامل: ۱۰۰

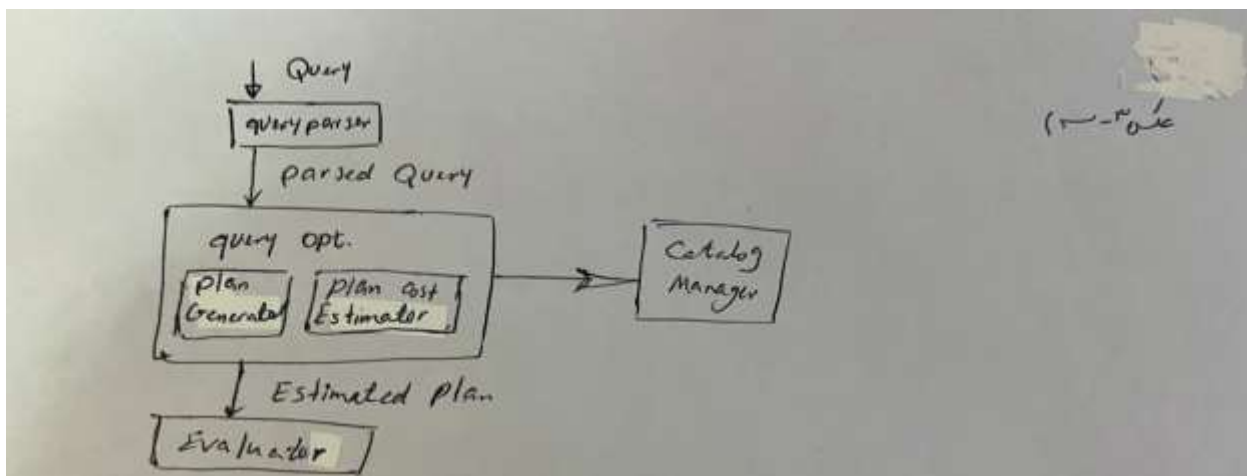
در این حالت، به صرفه‌تر است که تمام صفحات را به ترتیب در حافظه بارگذاری کرده و آنها را اسکن کنیم. شروط مورد نیاز را بر روی اطلاعات بر اساس حالت "on the fly" (در حین اجرا) بررسی می‌کنیم تا مطمئن شویم که شرایط مورد نظر برای هر رکورد برقرار است.

بنابراین، در این سناریو، بهترین روش اجرای کوئری اسکن کامل داده‌ها است به همراه بررسی شروط "on the fly"، زیرا هزینه آن کمتر از استفاده از هر دو نوع شاخص unclustered است.

(ب)

Query opt. در خودش دو کامپوننت دارد: plan generator و plan cost estimator. استیمیتور، هزینه ی پلن را تخمین میزند و Plan generator، یکسری پلن left deep تولید میکند عموماً.

تخمین زدن پلن بدین صورت است که برای داشتن کاتالوگ‌ها به کاتالوگ منیجر، مراجعه میکند و کاتالوگ relation‌ها را از آن دریافت می‌کند.



#### سوال (۴)

(الف)

• PONLJ :

○ اگر Enrolment به عنوان outer انتخاب شود و student inner باشد:

$$600 + 600 * 300 = 180600$$

○ هزینه در صورت اینکه Student به عنوان outer انتخاب شود و Enrolment به عنوان inner انتخاب شود .

$$300 + 300 * 600 = 180300$$

• INLJ :

$$\text{cost} = \text{Pages\_in\_r} + \text{Pages\_in\_r} * \text{tup\_per\_page} * \text{cost of finding matching S tuples}$$

○ هزینه در صورت اینکه Student به عنوان inner انتخاب شود:

$$600 + 600 * 150 * (3+1) = 360600$$

○ هزینه در صورت اینکه Enrolment به عنوان inner انتخاب شود:

$$300 + 300 * 20 * (1.2+10) = 67500$$

(ب)

با توجه به اینکه یک شاخص clustered بر روی دانش آموزان تعریف شده است، نیازی به مرتب‌سازی آنها نیست و هزینه مربوط به فرآیند مرتب‌سازی (external merge sort) را می‌توان با هزینه Enrollment جایگزین کرد. به عبارت دیگر، هزینه اصلی بخش مرتب‌سازی که برابر با ۶۰۰۰ بوده است، برای Enrollment استفاده می‌شود.

این بدین معناست که با توجه به وجود شاخص clustered، هزینه مرتب‌سازی اعمال بر داده‌ها به هزینه Enrollment منتقل می‌شود.

Handwritten notes showing the calculation of the cost of sorting:

$N = 600, B = 5$

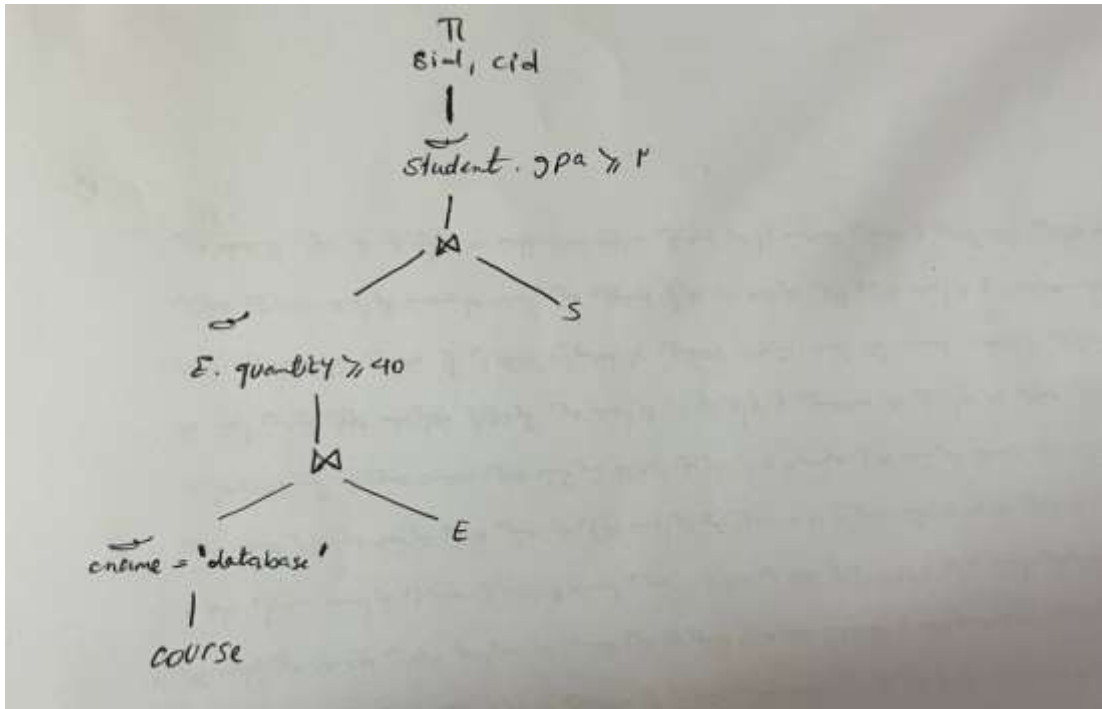
$cost = 2 \times 600 \times ((1 + \log_5 120) \times 6000)$

$I_N = (1 + \log_{B-1} \frac{N}{B})$

enrollment

external merge sort

سوال (۵)



مرحله اول:

در ابتدا، عملیات انتخاب را بر روی رابطه Course اجرا می‌کنیم، که این عمل با استفاده از شاخص درختی unclustered که بر روی فیلد cname تعریف شده، انجام می‌شود. دلیل این انتخاب این است که پس از اجرای این عمل انتخاب و شروع به کار با این رابطه، تعداد رکوردهای ما به ۲۰ واحد می‌رسد، که در عملیات انتخاب دیگری اینطور نیست.

مرحله دوم:

در این مرحله، وارد عملیات join بین رکوردهای انتخاب شده از جدول Course و جدول Enrollment می‌شویم. از آنجا که شاخص بر روی فیلد cid در جدول Enrollment تعریف شده است، ما از روش Nested Loop Join (INLJ) استفاده می‌کنیم و رکوردهای انتخاب شده از Course را به عنوان outer در نظر می‌گیریم. با توجه به اینکه برای هر درس ۱۰ ثبت‌نام وجود دارد، تعداد رکوردهای خروجی پس از انجام join برابر با  $20 * 10 = 200$  خواهد بود.

مرحله سوم:

در این مرحله، عملیات انتخاب (selection) را بر روی فیلد quantity اعمال می‌کنیم "on the fly". با توجه به اینکه مقادیر quantity یکنواخت بین ۱۰ و ۶۰ است، ما انتظار داریم که ۴۰٪ از رکوردها باقی بمانند، که برابر است با  $200 * 0.4 = 80$  رکورد.

مرحله چهارم:

در این مرحله، باقی مانده‌ها و جدول Student را join می‌کنیم. از روش Nested Loop Join (INLJ) استفاده می‌کنیم، زیرا شاخص بر روی فیلد sid در جدول Student وجود دارد. خروجی از عملیات selection قبلی را به عنوان outer در نظر می‌گیریم. با توجه به اینکه هر رکورد در outer متناظر با یک رکورد در Student است، تعداد رکوردهای خروجی همان ۸۰ رکورد است.

مرحله پنجم:

در این مرحله، عملیات selection را بر روی فیلد "gpa on the fly" بر روی خروجی مرحله قبل انجام می‌دهیم. با توجه به نمودار هیستوگرام، ۶۰٪ از داده‌ها دارای مقدار gpa بزرگتر یا مساوی ۲ هستند. بنابراین، خروجی نهایی ما شامل ۴۸ رکورد خواهد بود.

مرحله ششم:

در مرحله آخر، عملیات projection را انجام می‌دهیم تا به نتیجه نهایی برسیم.

هزینه‌ها:

$\text{traverse tree} + \text{read pages} = 3 + 20 = 23$   
 $\# \text{tuples} * (\text{traverse tree} + \text{read pages}) = 20 * (3 + 10) = 260$   
 $\# \text{tuples} * (\text{search hash bucket} + \text{read pages}) = 80 * (1.2 + 1) = 176$   
 $\text{Total cost} = 23 + 260 + 176 = 459$

سوال ۶)

الفـ

$$N = 1000, B = 8$$

$$N = 1000, B = 8$$

$$\# \text{ passes} = 1 + \log_{B-1} \left[ \frac{N}{B} \right] = 1 + \log_7 \left[ \frac{1000}{8} \right] = 5$$

cost of internal merge sort:  $2N \times (1 + \log) n$

# passo 1

# pass 7

+ pass 2:

دارد 592

Pass 3:

2. Explain

# pass 4.

$$\# \text{rvn: } [2|7] = 1$$

بار 10000

$$F = [B/b] = 1$$

$$\# \text{pages} = 1 + \lceil \log_2 \lceil N/2 \rceil \rceil \cdot 5$$

$\log 625 = 2.4$

$$3 < \log_F^{625} \leq 4$$

$$\rightarrow 5\% \text{ [B/W]} < 8.3 \rightarrow 4\% \text{ [B/W]} < 8.3$$

$$4/5 < b < 4/3$$

کسی به  $\frac{b=1}{\text{تکرار می شود}}$  pass دارد و اینم نهایت نه دارم!